

ELECTRO GO APPLICATION DEPLOYMENT ON MICROSOFT AZURE

Project Aim

The aim of this project is to deploy the Electro GO EV Charging Station Management Application on Microsoft Azure using a secure and scalable 2-Tier Architecture. The deployment focuses on hosting the ASP.NET Core application on an Ubuntu Virtual Machine and storing application data in Azure Database for MySQL Flexible Server while implementing Azure networking, security, and access management best practices.

The project also aims to provide hands-on experience with cloud infrastructure provisioning, application deployment, database connectivity, network security, load balancing, and identity management services available in Microsoft Azure.

Project Objectives

1. Deploy an ASP.NET Core 8 application on an Ubuntu Virtual Machine in Microsoft Azure.
2. Configure Azure Database for MySQL Flexible Server as the backend database.
3. Design and implement a secure 2-Tier Architecture.
4. Configure Virtual Network, Subnets, Route Tables, and Network Security Groups.
5. Implement secure communication between application and database layers.
6. Configure Azure Load Balancer for traffic distribution and high availability.
7. Enable Managed Identity and RBAC for secure access management.
8. Validate end-to-end connectivity between the application and database.
9. Follow Azure security and cloud infrastructure best practices.
10. Create a production-ready deployment environment for the Electro GO application.

Project Requirements

Functional Requirements

Application Requirements

- ASP.NET Core 8 Web Application

- Entity Framework Core
- MySQL Database Connectivity
- CRUD Operations for EV Charging Stations
- User Authentication and Authorization
- Application Configuration using appsettings.json

Database Requirements

- Azure Database for MySQL Flexible Server
- MySQL Version 8.0
- Private Access Configuration
- Database Backup and Recovery Support

Infrastructure Requirements

- Azure Resource Group
- Virtual Network (VNet)
- Public Subnet
- Private Subnet
- Network Security Groups (NSG)
- Route Table
- NAT Gateway
- Ubuntu Virtual Machine
- Azure Load Balancer
- Managed Identity
- Azure RBAC

Software Requirements

Component	Version
Operating System	Ubuntu Server 22.04 LTS
Application Framework	ASP.NET Core 8
Database	MySQL 8.0
SDK	.NET SDK 8.0
Cloud Platform	Microsoft Azure
File Transfer Tool	WinSCP
SSH Client	PuTTY / OpenSSH
IDE	Visual Studio Code
Source Control	Git & GitHub

Hardware Requirements

Local Machine

- Processor: Intel Core i5 or higher
- RAM: 8 GB Minimum (16 GB Recommended)
- Storage: 20 GB Free Space
- Internet Connection

Azure Virtual Machine

- VM Size: Standard_B2s
- vCPU: 2
- RAM: 4 GB
- Operating System: Ubuntu Server 22.04 LTS

Security Requirements

- SSH Access Restricted to Authorized Users
- Network Security Groups Configured
- Database Access Restricted to Virtual Network
- Managed Identity Enabled
- Role-Based Access Control (RBAC) Implemented
- Least Privilege Access Principle Followed
- Secure Database Credentials Management

Expected Outcome

Upon successful completion of the project:

- Electro GO Application will be accessible through Azure infrastructure.
- ASP.NET Core application will run on Ubuntu Virtual Machine.
- Azure Database for MySQL Flexible Server will store application data securely.
- Secure communication between application and database layers will be established.
- Azure networking and security services will be properly configured.
- The solution will demonstrate a complete production-style 2-Tier deployment architecture on Microsoft Azure.

Project Overview

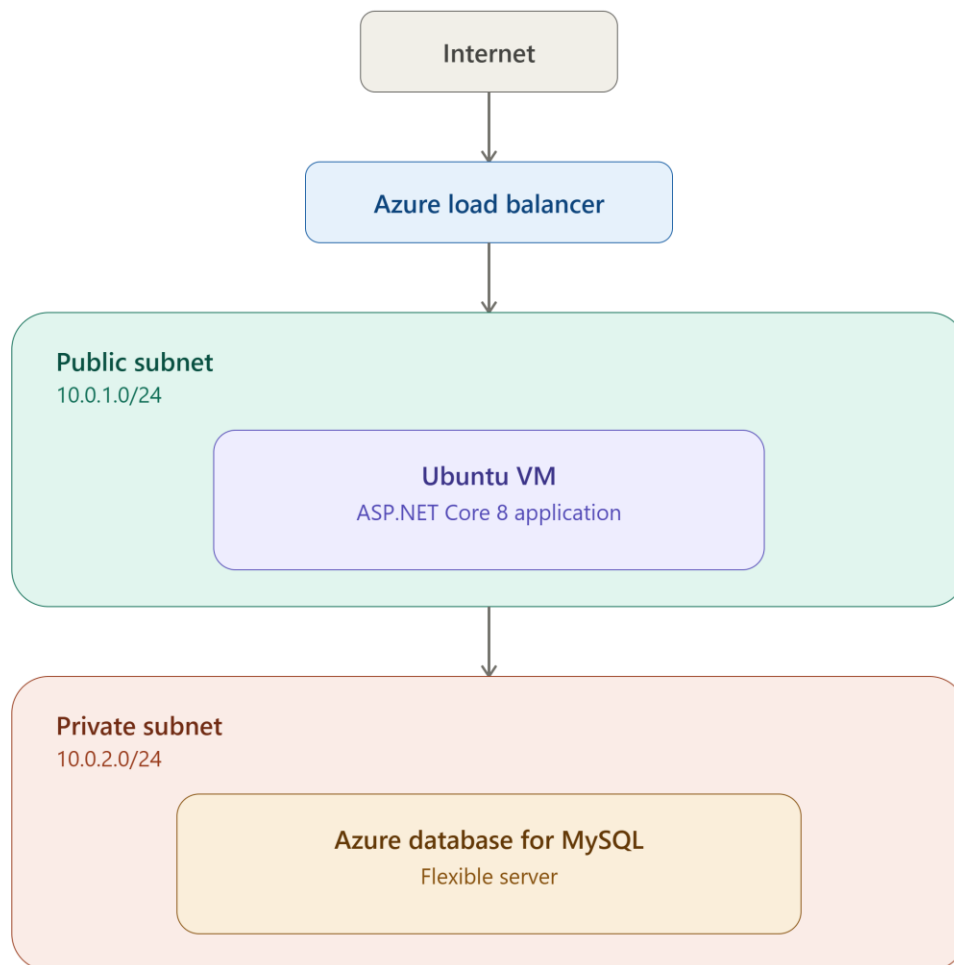
Electro GO is an EV Charging Station Management Application developed using ASP.NET Core 8 and MySQL. The objective of this project is to deploy the application on Microsoft Azure using a secure 2-Tier Architecture.

The deployment consists of:

- Application Tier hosted on Ubuntu Linux Virtual Machine
- Database Tier hosted on Azure Database for MySQL Flexible Server

Azure services such as Resource Groups, Virtual Networks, Subnets, Network Security Groups, Route Tables, NAT Gateway, Load Balancer, Managed Identity, and RBAC are used to ensure secure, scalable, and highly available infrastructure.

Architecture



STEP 1: CREATE RESOURCE GROUP

Aim

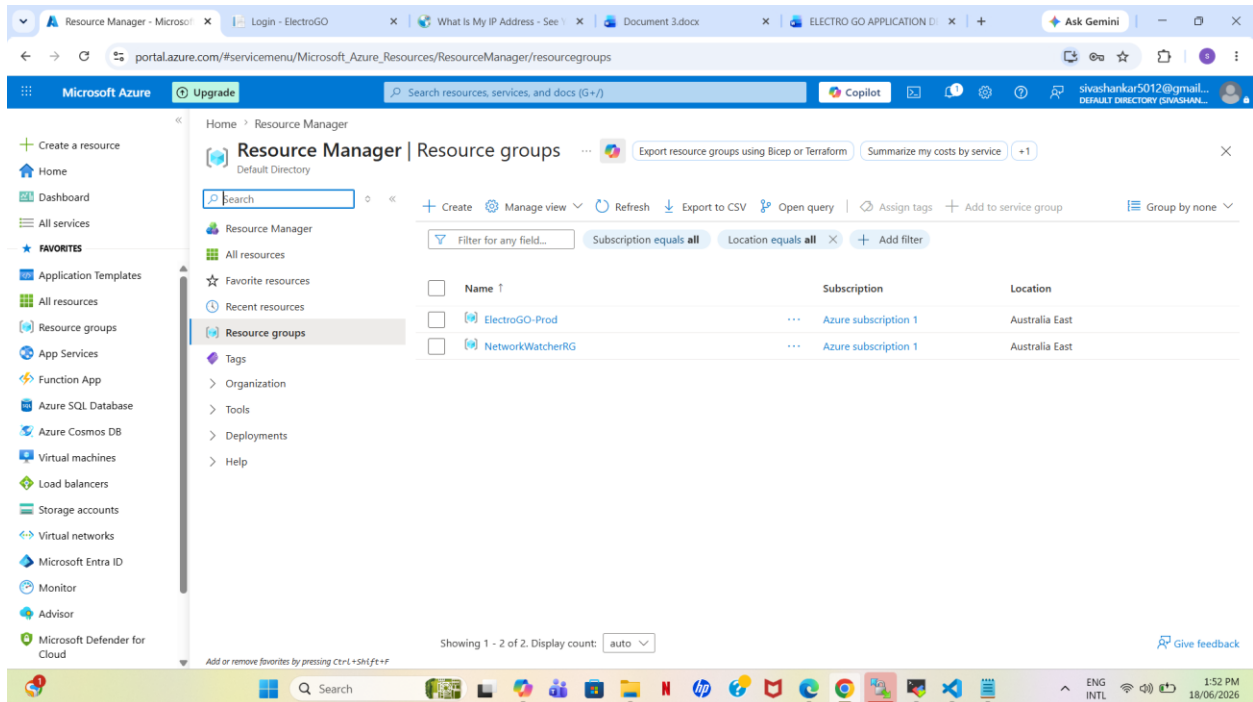
To create a Resource Group for organizing all Electro GO Azure resources.

Procedure

1. Login to Azure Portal.
2. Search Resource Groups.

3. Click Create.
4. Resource Group Name:
ElectroGO-Prod
5. Region:
Australia East
6. Click Review + Create.
7. Click Create.

Screenshot



Outcome

Resource Group was successfully created.

STEP 2: CREATE VIRTUAL NETWORK

Aim

To create a Virtual Network for secure communication between Azure resources.

Procedure

1. Navigate to Virtual Networks.
2. Click Create.

3. Name:

- vnet-electrogo

4. Address Space:

- 10.0.0.0/16

5.Create Public Subnet:

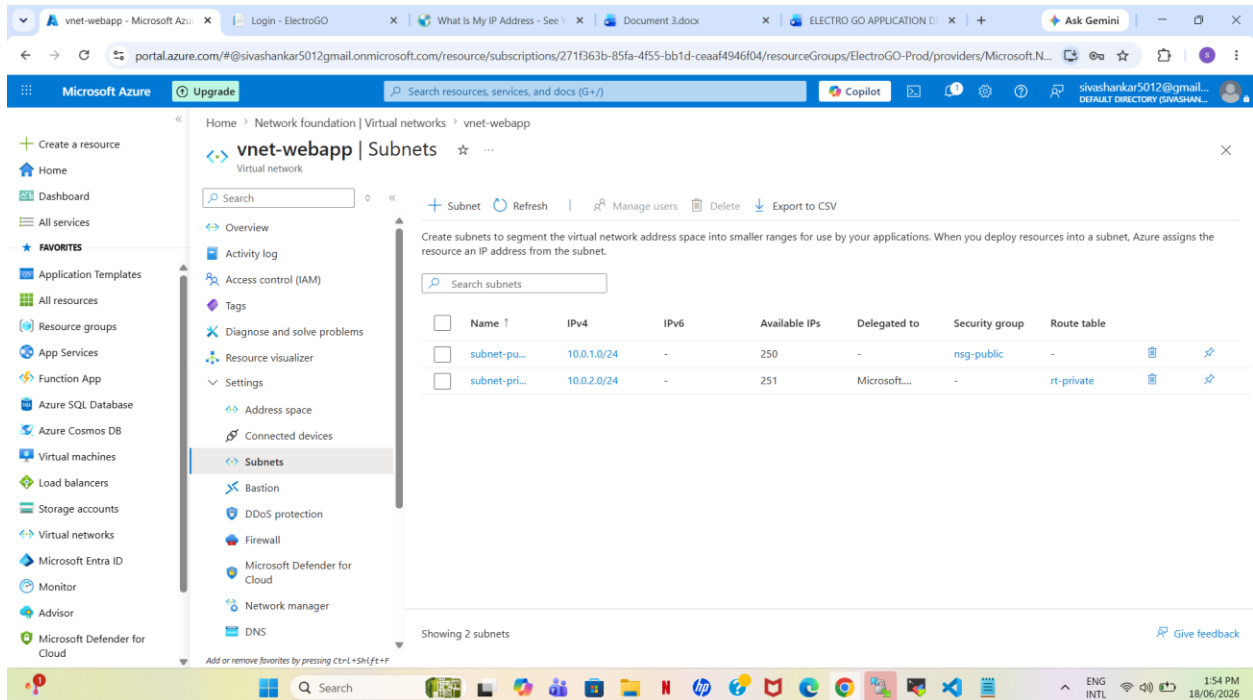
- Name:
- subnet-public
- CIDR:
- 10.0.1.0/24

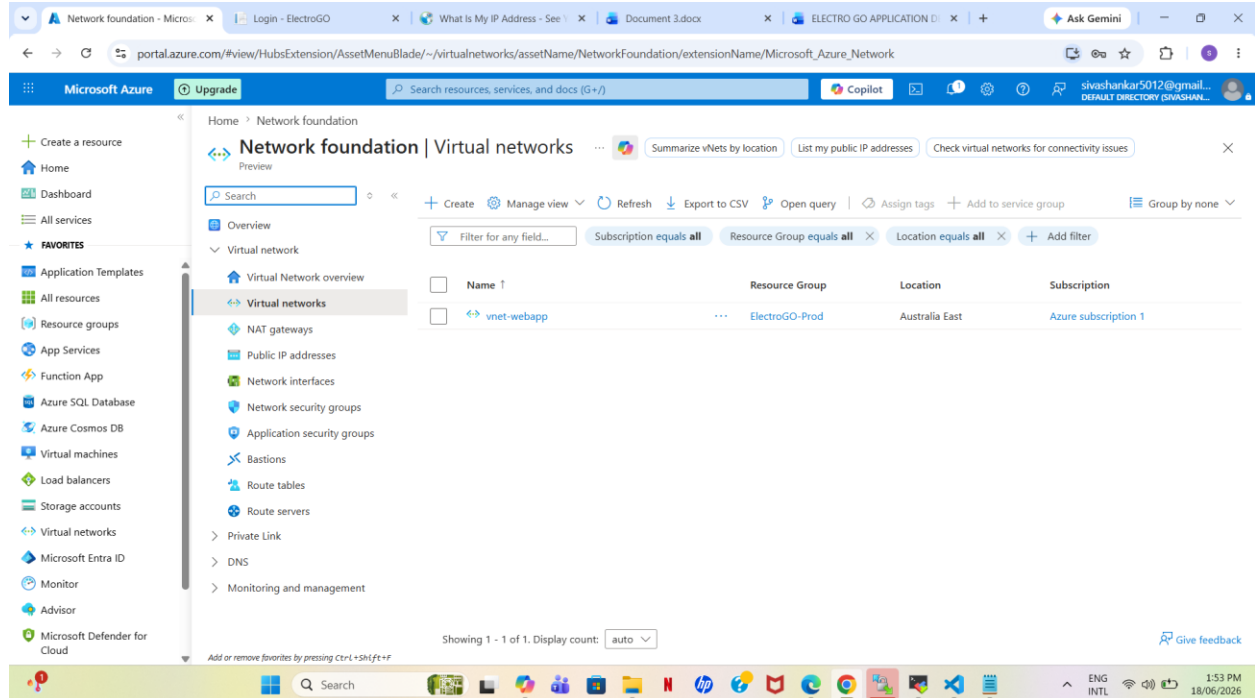
6.Create Private Subnet:

- Name:
- subnet-private
- CIDR:
- 10.0.2.0/24

7.Review and Create.

Screenshot





Outcome

Virtual Network and Subnets created successfully.

STEP 3: CREATE NAT GATEWAY

Aim

To provide outbound internet access for private subnet resources.

Procedure

1. Navigate to NAT Gateway.
2. Click Create.
3. Name:
 - natgw-electrogo
4. Create Public IP.
5. Associate with subnet-private.
6. Review and Create.

Screenshot

The first screenshot shows the 'Network foundation | NAT gateways' overview page in the Azure portal. The left sidebar contains the navigation menu with 'NAT gateways' selected. The main content area displays a table with one NAT gateway resource:

Name	Resource Group	Location	Subscription	Type
NATGW-webapp	ElectroGO-Prod	Australia East	Azure subscription 1	NAT gateway

The second screenshot shows the 'NATGW-webapp | Networking' configuration page. It displays the 'Virtual network' dropdown set to 'vnet-webapp' and the 'Select specific subnets' dropdown set to 'subnet-private (10.0.2.0/24)'. The page also includes a 'Settings' section with options for Outbound IP, Configuration, Properties, Locks, Monitoring, Automation, and Help.

Outcome

Private subnet resources can access the internet securely.

STEP 4: CREATE ROUTE TABLE

Aim

To manage traffic routing for private subnet resources.

Procedure

1. Navigate to Route Tables.
2. Click Create.
3. Name:
 - rt-private
4. Deploy Route Table.
5. Associate with subnet-private.

Screenshot

The first screenshot shows the 'Network foundation | Route tables' overview page in the Azure portal. The left sidebar contains navigation links for 'Create a resource', 'Home', 'Dashboard', 'All services', and 'FAVORITES'. The main content area displays a table of route tables. The table has columns for 'Name', 'Resource Group', 'Location', and 'Subscription'. One route table is listed: 'rt-private' in the 'ElectroGO-Prod' resource group, located in 'Australia East' under 'Azure subscription 1'.

Name	Resource Group	Location	Subscription
rt-private	ElectroGO-Prod	Australia East	Azure subscription 1

The second screenshot shows the 'rt-private' route table details page. The left sidebar is the same. The main content area shows the 'Overview' tab with details for the route table. The 'Essentials' section lists: Resource group: ElectroGO-Prod, Location: Australia East, Subscription: Azure subscription 1, and Subscription ID: 271f363b-85fa-4f55-bb1d-aeaaf4946f04. The 'Routes' section shows a search bar and a table with columns: Name, Address prefix, Next hop type, and Next hop IP address. The table is empty, showing 'No results.' The 'Subnets' section shows a search bar and a table with columns: Name, Address range, Virtual network, Security group, and an ellipsis. One subnet is listed: 'subnet-private' with address range '10.0.2.0/24' and virtual network 'vnet-webapp'.

Name	Address range	Virtual network	Security group
subnet-private	10.0.2.0/24	vnet-webapp	-

Outcome

Traffic routing configured successfully.

Katta siva shankar

Sivashankar9929@gmail.com

STEP 5: CREATE NETWORK SECURITY GROUPS

Aim

To secure network communication.

Procedure

Create Public NSG:

- Name:
 - nsg-public
- Inbound Rules:
 - SSH – 22
 - HTTP – 80
 - HTTPS – 443
 - ASP.NET Core – 5000
- Associate with subnet-public.
- Create Private NSG:
 - Name:
 - nsg-private
 - Inbound Rule:
 - MySQL – 3306

Source:

Virtual Network

Associate with subnet-private.

Screenshot

The image displays two screenshots of the Microsoft Azure portal interface, showing the configuration and overview of Network Security Groups (NSGs).

Top Screenshot: Network foundation | Network security groups

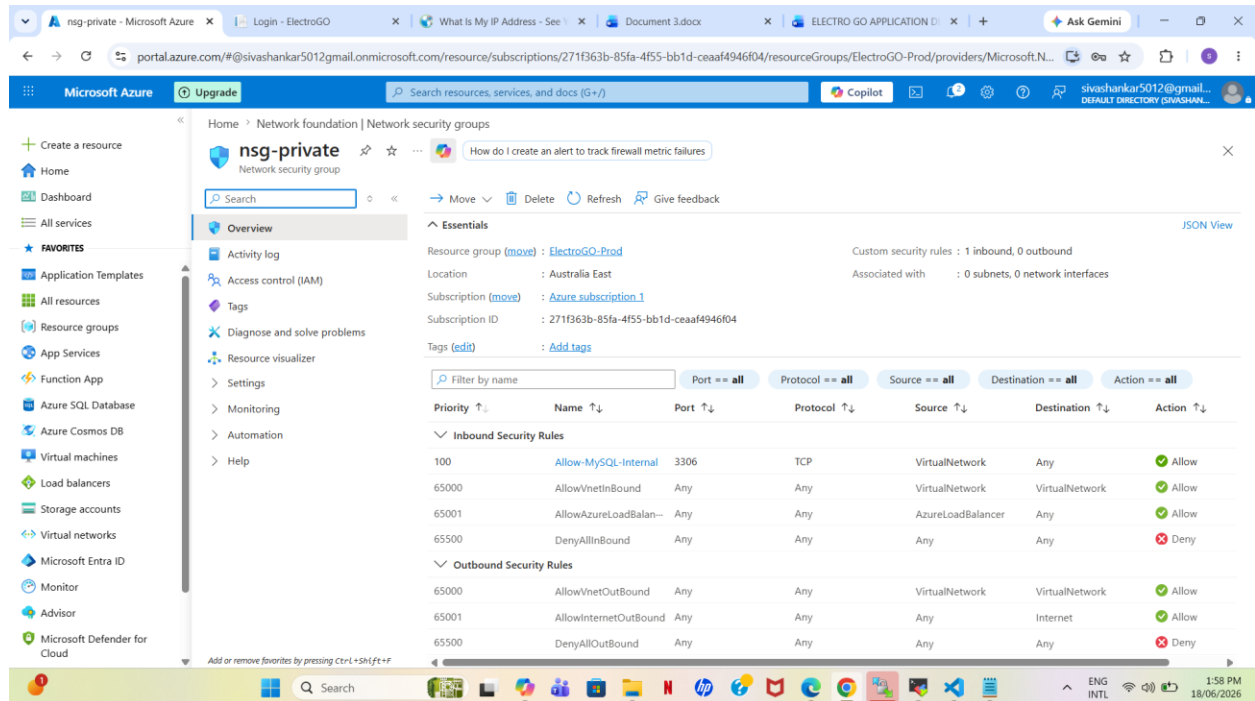
This view shows the 'Network foundation | Network security groups' page. The left sidebar lists various Azure services, including 'Network security groups'. The main content area displays a table of NSGs:

Name	Resource Group	Location	Subscription	Flow log
nsg-private	ElectroGO-Prod	Australia East	Azure subscription 1	
nsg-public	ElectroGO-Prod	Australia East	Azure subscription 1	

Bottom Screenshot: nsg-public Overview

This view shows the 'nsg-public' Network Security Group overview. The left sidebar lists various Azure services, including 'Network security groups'. The main content area displays the 'Overview' tab, showing the NSG's configuration and a table of security rules:

Priority	Name	Port	Protocol	Source	Destination	Action
100	Allow-SSH	22	TCP	Any	Any	Allow
110	Allow-HTTP	80	TCP	Any	Any	Allow
120	Allow-HTTPS	443	TCP	Any	Any	Allow
130	ASP.NET	5000	TCP	Any	Any	Allow
150	sql	3306	TCP	4.254.27.32	Any	Allow
65000	AllowVnetInBound	Any	Any	VirtualNetwork	VirtualNetwork	Allow
65001	AllowAzureLoadBalanc--	Any	Any	AzureLoadBalancer	Any	Allow
65500	DenyAllInBound	Any	Any	Any	Any	Deny
65000	AllowVnetOutBound	Any	Any	VirtualNetwork	VirtualNetwork	Allow
65001	AllowInternetOutBound	Any	Any	Any	Internet	Allow
65500	DenyAllOutBound	Any	Any	Any	Any	Deny



Outcome

Network traffic secured successfully.

STEP 6: CREATE UBUNTU VIRTUAL MACHINE

Aim

To provision Ubuntu Linux VM for hosting Electro GO application.

Procedure

1. Navigate to Virtual Machines.
2. Click Create.
3. VM Name:
 - vm-electrogo-app-01
4. Image:
 - Ubuntu Server 22.04 LTS
5. Size:
 - Standard_B2s
6. Authentication:
 - SSH Key
7. Enable Public IP.

Katta siva shankar

Sivashankar9929@gmail.com

8. Enable Managed Identity.
9. Create VM.

Screenshot

The top screenshot shows the Microsoft Azure portal's 'Virtual machines' overview page. The left sidebar contains navigation options like 'Home', 'Dashboard', 'All services', and 'FAVORITES'. The main content area displays a table of virtual machines. Below the table, there are filters and a 'Showing 1 - 1 of 1' indicator.

Name	Subscription	Resource Group	Private IP address	Public IP address	Location	Status	Owner
WebApp-Server	Azure subscri...	ElectroGO-Prod	10.0.1.4	4.254.27.32	Australia East	Running	Lir

The bottom screenshot shows the 'WebApp-Server' VM details page. The left sidebar is the same as the top screenshot. The main content area displays the VM's configuration details, including 'Virtual machine', 'Networking', 'Size', and 'Source image details'.

Virtual machine details:

- Computer name: WebApp-Server
- Operating system: Linux (ubuntu 24.04)
- VM generation: v2
- VM architecture: x64
- Agent status: Ready
- Agent version: 2.15.2.1
- Hibernation: Disabled
- Host group: -
- Host: -
- Proximity placement group: -
- Colocation status: N/A
- Capacity reservation group: -
- Disk controller type: SCSI

Networking details:

- Public IP address: 4.254.27.32 (Network interface: webapp-server323)
- Public IP address (IPv6): -
- Private IP address: 10.0.1.4
- Private IP address (IPv6): -
- Virtual network/subnet: vnet-webapp/subnet-public
- DNS name: Configure

Size details:

- Size: Standard D2s v3
- vCPUs: 2
- RAM: 8 GiB
- Threads per core: 2

Source image details:

- Source image publisher: canonical

Outcome

Ubuntu VM deployed successfully.

STEP 7: INSTALL .NET 8 SDK ON UBUNTU VM

Aim

To install ASP.NET Core runtime and .NET SDK.

Procedure

Connect to VM:

- `sudo apt update`
- `sudo apt upgrade -y`
- `wget https://packages.microsoft.com/config/ubuntu/22.04/packages-microsoft-prod.deb`
- `sudo dpkg -i packages-microsoft-prod.deb`
- `sudo apt update`
- `sudo apt install -y dotnet-sdk-8.0`
- Verify Installation:
- `dotnet --version`
- `dotnet --list-sdks`
- `dotnet --list-runtimes`

Screenshot

To run a command as administrator (user "root"), use "sudo <command>".
See "man sudo_root" for details.

```
Azureuser@WebApp-Server:~$ sudo su
root@WebApp-Server:/home/Azureuser# apt update
Hit:1 http://azure.archive.ubuntu.com/ubuntu noble InRelease
Get:2 http://azure.archive.ubuntu.com/ubuntu noble-updates InRelease [126 kB]
Get:3 http://azure.archive.ubuntu.com/ubuntu noble-backports InRelease [126 kB]
Get:4 http://azure.archive.ubuntu.com/ubuntu noble-security InRelease [126 kB]
Get:5 http://azure.archive.ubuntu.com/ubuntu noble/universe amd64 Packages [15.0 MB]
Get:6 http://azure.archive.ubuntu.com/ubuntu noble/universe Translation-en [5982 kB]
Get:7 http://azure.archive.ubuntu.com/ubuntu noble/universe amd64 Components [3871 kB]
Get:8 http://azure.archive.ubuntu.com/ubuntu noble/universe amd64 c-n-f Metadata [301 kB]
Get:9 http://azure.archive.ubuntu.com/ubuntu noble/multiverse amd64 Packages [269 kB]
Get:10 http://azure.archive.ubuntu.com/ubuntu noble/multiverse Translation-en [118 kB]
Get:11 http://azure.archive.ubuntu.com/ubuntu noble/multiverse amd64 Components [35.0 kB]
Get:12 http://azure.archive.ubuntu.com/ubuntu noble/multiverse amd64 c-n-f Metadata [8328 B]
Get:13 http://azure.archive.ubuntu.com/ubuntu noble-updates/main amd64 Packages [1030 kB]
Get:14 http://azure.archive.ubuntu.com/ubuntu noble-updates/main Translation-en [258 kB]
Get:15 http://azure.archive.ubuntu.com/ubuntu noble-updates/main amd64 Components [181 kB]
Get:16 http://azure.archive.ubuntu.com/ubuntu noble-updates/main amd64 c-n-f Metadata [17.4 kB]
Get:17 http://azure.archive.ubuntu.com/ubuntu noble-updates/universe amd64 Packages [1655 kB]
Get:18 http://azure.archive.ubuntu.com/ubuntu noble-updates/univer
```

```

root@WebApp-Server:/home/Azureuser# sudo apt upgrade -y
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
Calculating upgrade... Done
The following packages were automatically installed and are no longer required:
  libfwupd2 libgusb2
Use 'sudo apt autoremove' to remove them.
The following NEW packages will be installed:
  libdrm-amdgpu1 libfwupd3
The following packages will be upgraded:
  ca-certificates fwupd inetutils-telnet libdrm-common libdrm2 libjcat1 libnss-systemd libpam-systemd libssl3t64 libsystemd-shared
  libsystemd0 libudev1 libxmlb2 openssl rsync systemd systemd-dev systemd-resolved systemd-sysv telnet udev vim vim-common
  vim-runtime vim-tiny xxd
26 upgraded, 2 newly installed, 0 to remove and 0 not upgraded.
23 standard LTS security updates
Need to get 29.3 MB of archives.
After this operation, 5159 kB of additional disk space will be used.
Get:1 http://azure.archive.ubuntu.com/ubuntu noble-updates/main amd64 libnss-systemd amd64 255.4-1ubuntu8.16 [159 kB]
Get:2 http://azure.archive.ubuntu.com/ubuntu noble-updates/main amd64 systemd-dev all 255.4-1ubuntu8.16 [106 kB]
Get:3 http://azure.archive.ubuntu.com/ubuntu noble-updates/main amd64 systemd-resolved amd64 255.4-1ubuntu8.16 [296 kB]
Get:4 http://azure.archive.ubuntu.com/ubuntu noble-updates/main amd64 libsystemd-shared amd64 255.4-1ubuntu8.16 [2076 kB]
Get:5 http://azure.archive.ubuntu.com/ubuntu noble-updates/main amd64 libsystemd0 amd64 255.4-1ubuntu8.16 [431 kB]
Get:6 http://azure.archive.ubuntu.com/ubuntu noble-updates/main amd64 systemd-sysv amd64 255.4-1ubuntu8.16 [11.9 kB]
Get:7 http://azure.archive.ubuntu.com/ubuntu noble-updates/main amd64 libpam-systemd amd64 255.4-1ubuntu8.16 [235 kB]
Get:8 http://azure.archive.ubuntu.com/ubuntu noble-updates/main amd64 systemd amd64 255.4-1ubuntu8.16 [3475 kB]
Get:9 http://azure.archive.ubuntu.com/ubuntu noble-updates/main amd64 udev amd64 255.4-1ubuntu8.16 [1875 kB]
Get:10 http://azure.archive.ubuntu.com/ubuntu noble-updates/main amd64 libudev1 amd64 255.4-1ubuntu8.16 [177 kB]
Get:11 http://azure.archive.ubuntu.com/ubuntu noble-updates/main amd64 libssl3t64 amd64 3.0.13-0ubuntu3.11 [1942 kB]
Get:12 http://azure.archive.ubuntu.com/ubuntu noble-updates/main amd64 rsync amd64 3.2.7-1ubuntu1.5 [443 kB]
Get:13 http://azure.archive.ubuntu.com/ubuntu noble-updates/main amd64 openssl amd64 3.0.13-0ubuntu3.11 [1003 kB]
Get:14 http://azure.archive.ubuntu.com/ubuntu noble-updates/main amd64 ca-certificates all 20260601~24.04.1 [139 kB]
Get:15 http://azure.archive.ubuntu.com/ubuntu noble-updates/main amd64 vim amd64 2:9.1.0016-1ubuntu7.15 [1883 kB]
Get:16 http://azure.archive.ubuntu.com/ubuntu noble-updates/main amd64 vim-common all 2:9.1.0016-1ubuntu7.15 [387 kB]
Get:17 http://azure.archive.ubuntu.com/ubuntu noble-updates/main amd64 vim-tiny amd64 2:9.1.0016-1ubuntu7.15 [805 kB]
Get:18 http://azure.archive.ubuntu.com/ubuntu noble-updates/main amd64 vim-runtime all 2:9.1.0016-1ubuntu7.15 [7282 kB]
Get:19 http://azure.archive.ubuntu.com/ubuntu noble-updates/main amd64 xxd amd64 2:9.1.0016-1ubuntu7.15 [65.0 kB]
Get:20 http://azure.archive.ubuntu.com/ubuntu noble-updates/main amd64 inetutils-telnet amd64 2:2.5-3ubuntu4.2 [99.5 kB]

root@WebApp-Server:/home/Azureuser# wget https://packages.microsoft.com/config/ubuntu/24.04/packages-microsoft-prod.deb
--2026-06-17 09:53:25-- https://packages.microsoft.com/config/ubuntu/24.04/packages-microsoft-prod.deb
Resolving packages.microsoft.com (packages.microsoft.com)... 13.107.246.31, 13.107.213.31, 2620:1ec:46::31, ...
Connecting to packages.microsoft.com (packages.microsoft.com)|13.107.246.31|:443... connected.
HTTP request sent, awaiting response... 200 OK
Length: 4288 (4.2K) [application/octet-stream]
Saving to: 'packages-microsoft-prod.deb'

packages-microsoft-prod.deb 100%[=====] 4.19K --.-KB/s in 0s

2026-06-17 09:53:25 (1.43 GB/s) - 'packages-microsoft-prod.deb' saved [4288/4288]

root@WebApp-Server:/home/Azureuser# wget https://packages.microsoft.com/config/ubuntu/22.04/packages-microsoft-prod.deb
--2026-06-17 09:53:40-- https://packages.microsoft.com/config/ubuntu/22.04/packages-microsoft-prod.deb
Resolving packages.microsoft.com (packages.microsoft.com)... 13.107.246.31, 13.107.213.31, 2620:1ec:bdf::31, ...
Connecting to packages.microsoft.com (packages.microsoft.com)|13.107.246.31|:443... connected.
HTTP request sent, awaiting response... 200 OK
Length: 4070 (4.0K) [application/octet-stream]
Saving to: 'packages-microsoft-prod.deb.1'

packages-microsoft-prod.deb.1 100%[=====] 3.97K --.-KB/s in 0s

2026-06-17 09:53:40 (1.22 GB/s) - 'packages-microsoft-prod.deb.1' saved [4070/4070]

root@WebApp-Server:/home/Azureuser# sudo dpkg -i packages-microsoft-prod.deb
Selecting previously unselected package packages-microsoft-prod.
(Reading database ... 69443 files and directories currently installed.)
Preparing to unpack packages-microsoft-prod.deb ...
Unpacking packages-microsoft-prod (1.2-ubuntu24.04) ...
Setting up packages-microsoft-prod (1.2-ubuntu24.04) ...
root@WebApp-Server:/home/Azureuser# sudo apt update
Get:1 https://packages.microsoft.com/ubuntu/24.04/prod noble InRelease [3600 B]
Get:2 https://packages.microsoft.com/ubuntu/24.04/prod noble/main all Packages [643 B]
Hit:3 http://azure.archive.ubuntu.com/ubuntu noble InRelease
Hit:4 http://azure.archive.ubuntu.com/ubuntu noble-updates InRelease
Hit:5 http://azure.archive.ubuntu.com/ubuntu noble-backports InRelease
Hit:6 http://azure.archive.ubuntu.com/ubuntu noble-security InRelease
Get:7 https://packages.microsoft.com/ubuntu/24.04/prod noble/main amd64 Packages [176 kB]
Get:8 https://packages.microsoft.com/ubuntu/24.04/prod noble/main armhf Packages [11.6 kB]

```

```

root@WebApp-Server:/home/Azureuser# wget https://packages.microsoft.com/config/ubuntu/22.04/packages-microsoft-prod.deb
--2026-06-17 09:53:40-- https://packages.microsoft.com/config/ubuntu/22.04/packages-microsoft-prod.deb
Resolving packages.microsoft.com (packages.microsoft.com)... 13.107.246.31, 13.107.213.31, 2620:1ec:bdf::31, ...
Connecting to packages.microsoft.com (packages.microsoft.com)[13.107.246.31]:443... connected.
HTTP request sent, awaiting response... 200 OK
Length: 4070 (4.0K) [application/octet-stream]
Saving to: 'packages-microsoft-prod.deb.1'

packages-microsoft-prod.deb.1 100%[=====] 3.97K --.-KB/s in 0s

2026-06-17 09:53:40 (1.22 GB/s) - 'packages-microsoft-prod.deb.1' saved [4070/4070]

root@WebApp-Server:/home/Azureuser# sudo dpkg -i packages-microsoft-prod.deb
Selecting previously unselected package packages-microsoft-prod.
(Reading database ... 69443 files and directories currently installed.)
Preparing to unpack packages-microsoft-prod.deb ...
Unpacking packages-microsoft-prod (1.2-ubuntu24.04) ...
Setting up packages-microsoft-prod (1.2-ubuntu24.04) ...
root@WebApp-Server:/home/Azureuser# sudo apt update
Get:1 https://packages.microsoft.com/ubuntu/24.04/prod noble InRelease [3600 B]
Get:2 https://packages.microsoft.com/ubuntu/24.04/prod noble/main all Packages [643 B]
Hit:3 http://azure.archive.ubuntu.com/ubuntu noble InRelease
Hit:4 http://azure.archive.ubuntu.com/ubuntu noble-updates InRelease
Hit:5 http://azure.archive.ubuntu.com/ubuntu noble-backports InRelease
Hit:6 http://azure.archive.ubuntu.com/ubuntu noble-security InRelease
Get:7 https://packages.microsoft.com/ubuntu/24.04/prod noble/main amd64 Packages [176 kB]
Get:8 https://packages.microsoft.com/ubuntu/24.04/prod noble/main armhf Packages [11.6 kB]
Get:9 https://packages.microsoft.com/ubuntu/24.04/prod noble/main arm64 Packages [146 kB]
Fetched 337 kB in 1s (476 kB/s)
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
All packages are up to date.

root@WebApp-Server:/home/Azureuser# dotnet --version
8.0.128
root@WebApp-Server:/home/Azureuser# dotnet --info
.NET SDK:
  Version:           8.0.128
  Commit:            d091372eb2
  Workload version:  8.0.100-manifests.f89b5038

Runtime Environment:
  OS Name:           ubuntu
  OS Version:        24.04
  OS Platform:       Linux
  RID:               ubuntu.24.04-x64
  Base Path:         /usr/lib/dotnet/sdk/8.0.128/

.NET workloads installed:
  Workload version:  8.0.100-manifests.f89b5038
  There are no installed workloads to display.

Host:
  Version:           8.0.28
  Architecture:      x64
  Commit:            46295af582

```

Outcome

.NET 8 SDK installed successfully.

STEP 8: DEPLOY ASP.NET CORE APPLICATION

Aim

To deploy Electro GO application to Ubuntu VM.

Procedure

On Local Machine:

Katta siva shankar

Sivashankar9929@gmail.com

- dotnet publish -c Release -o publish

Using WinSCP:

- Upload publish folder to:
- /home/azureuser/electrogo

Run Application:

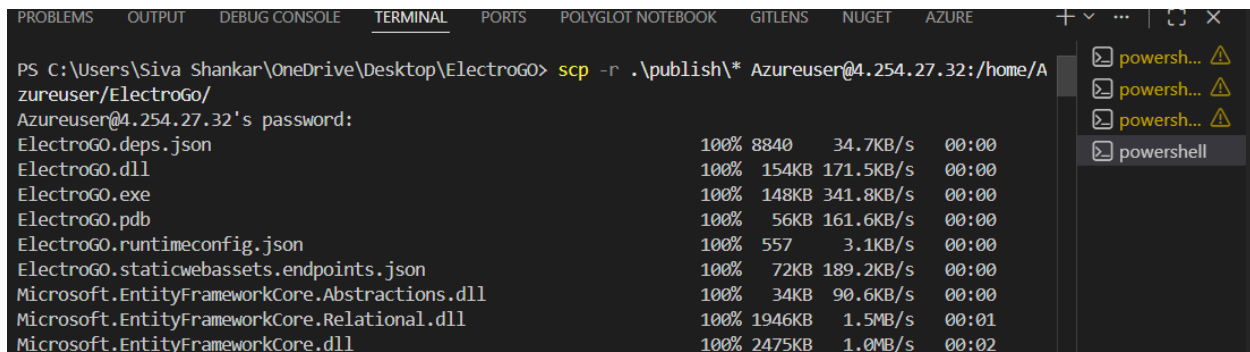
- cd /home/azureuser/electrogo

dotnet ElectroGO.Web.dll

Verify:

- url <http://localhost:5000>

Screenshot



```
PS C:\Users\Siva Shankar\OneDrive\Desktop\ElectroGO> scp -r .\publish\* Azureuser@4.254.27.32:/home/Azureuser/ElectroGo/
Azureuser@4.254.27.32's password:
ElectroGO.deps.json      100% 8840    34.7KB/s   00:00
ElectroGO.dll            100% 154KB   171.5KB/s  00:00
ElectroGO.exe            100% 148KB   341.8KB/s  00:00
ElectroGO.pdb            100% 56KB    161.6KB/s  00:00
ElectroGO.runtimeconfig.json 100% 557     3.1KB/s   00:00
ElectroGO.staticwebassets.endpoints.json 100% 72KB    189.2KB/s  00:00
Microsoft.EntityFrameworkCore.Abstractions.dll 100% 34KB    90.6KB/s   00:00
Microsoft.EntityFrameworkCore.Relational.dll 100% 1946KB  1.5MB/s    00:01
Microsoft.EntityFrameworkCore.dll 100% 2475KB  1.0MB/s    00:02
```

ElectroGo - Azureuser@4.254.27.32 - WinSCP

Local Mark Files Commands Tabs Options Remote Help

Synchronize Queue Transfer Settings: Default

Azureuser@4.254.27.32 x New Tab

Desktop

Upload Edit Properties New

C:\Users\Siva Shankar\OneDrive\Desktop\ElectroGO\publish\

Name	Size	Type	Changed
..		Parent directory	16/06/2026 9:27:27 PM
runtimes		File folder	16/06/2026 9:27:03 PM
wwwroot		File folder	16/06/2026 9:27:03 PM
appsettings.Developm...	1 KB	JSON Source File	16/06/2026 2:34:17 PM
appsettings.json	1 KB	JSON Source File	18/06/2026 8:41:21 AM
ElectroGO.deps.json	9 KB	JSON Source File	18/06/2026 9:12:23 AM
ElectroGO.dll	154 KB	Application extensi...	18/06/2026 9:12:22 AM
ElectroGO.exe	148 KB	Application	18/06/2026 9:12:22 AM
ElectroGO.pdb	57 KB	VisualStudio.pdb.d...	18/06/2026 9:12:22 AM
ElectroGO.runtimecon...	1 KB	JSON Source File	18/06/2026 9:12:23 AM
ElectroGO.staticwebas...	73 KB	JSON Source File	16/06/2026 9:27:03 PM
Microsoft.EntityFrame...	35 KB	Application extensi...	18/09/2024 5:19:08 AM
Microsoft.EntityFrame...	2,475 KB	Application extensi...	18/09/2024 5:19:44 AM
Microsoft.EntityFrame...	1,946 KB	Application extensi...	18/09/2024 5:20:04 AM
Microsoft.Extensions.C...	45 KB	Application extensi...	17/09/2024 4:40:10 AM
Microsoft.Extensions.D...	63 KB	Application extensi...	17/09/2024 4:39:34 AM
Microsoft.Extensions.D...	91 KB	Application extensi...	17/09/2024 4:39:36 AM
Microsoft.Extensions.L...	64 KB	Application extensi...	17/09/2024 4:39:58 AM
Microsoft.Extensions.L...	50 KB	Application extensi...	17/09/2024 4:40:06 AM
Microsoft.Extensions.O...	64 KB	Application extensi...	18/01/2024 8:35:26 AM
MySQLConnector.dll	810 KB	Application extensi...	20/01/2024 6:24:02 PM
Pomelo.EntityFramew...	494 KB	Application extensi...	16/03/2024 3:20:50 PM
web.config	1 KB	CONFIG File	18/06/2026 9:12:23 AM

4.77 MB of 6.41 MB in 14 of 22

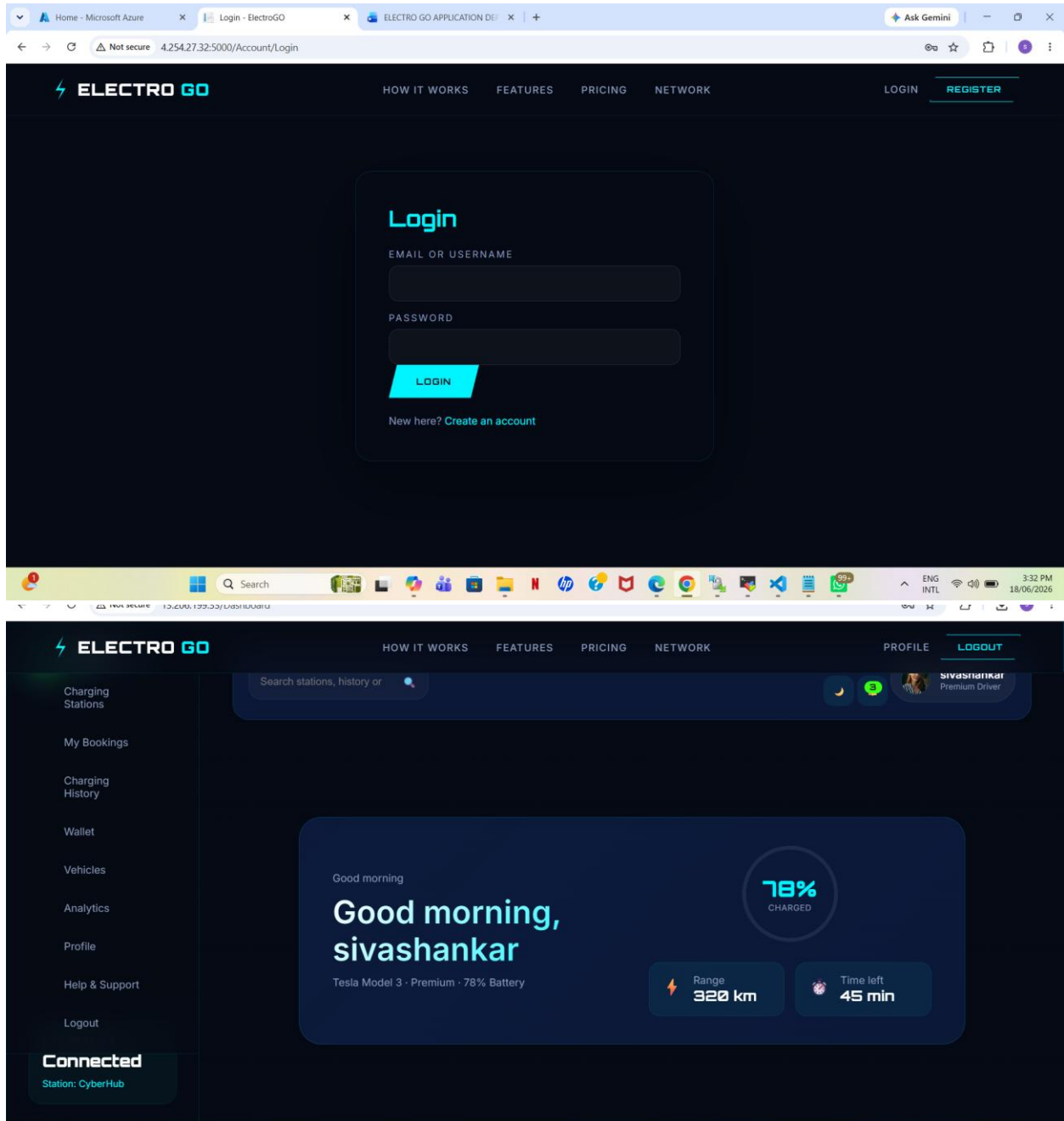
/home/Azureuser/ElectroGo/

Name	Size	Changed	Rights	Owner
Views		17/06/2026 4:42:31 PM	rw-rw-r--	Azureus...
wwwroot		17/06/2026 3:31:46 PM	rw-rw-r--	Azureus...
app.log	6 KB	18/06/2026 11:15:27 AM	rw-rw-r--	Azureus...
appsettings.Developm...	1 KB	18/06/2026 9:14:53 AM	rw-rw-r--	Azureus...
appsettings.json	1 KB	18/06/2026 11:06:08 AM	rw-rw-r--	Azureus...
ElectroGO.db	12 KB	17/06/2026 10:48:14 PM	rw-rw-r--	Azureus...
ElectroGO.deps.json	9 KB	18/06/2026 9:14:34 AM	rw-rw-r--	Azureus...
ElectroGO.dll	154 KB	18/06/2026 9:14:36 AM	rw-rw-r--	Azureus...
ElectroGO.exe	148 KB	18/06/2026 9:14:37 AM	rw-rw-r--	Azureus...
ElectroGO.pdb	57 KB	18/06/2026 9:14:38 AM	rw-rw-r--	Azureus...
ElectroGO.runtimecon...	1 KB	18/06/2026 9:14:38 AM	rw-rw-r--	Azureus...
ElectroGO.staticwebas...	73 KB	18/06/2026 9:14:40 AM	rw-rw-r--	Azureus...
HomeController.cs	1 KB	17/06/2026 4:42:09 PM	rw-rw-r--	Azureus...
Microsoft.Data.Sqlite...	170 KB	17/06/2026 5:18:38 PM	rw-rw-r--	Azureus...
Microsoft.EntityFrame...	35 KB	18/06/2026 9:14:40 AM	rw-rw-r--	Azureus...
Microsoft.EntityFrame...	2,475 KB	18/06/2026 9:14:45 AM	rw-rw-r--	Azureus...
Microsoft.EntityFrame...	1,946 KB	18/06/2026 9:14:43 AM	rw-rw-r--	Azureus...
Microsoft.EntityFrame...	250 KB	17/06/2026 5:18:43 PM	rw-rw-r--	Azureus...
Microsoft.Extensions.C...	45 KB	18/06/2026 9:14:46 AM	rw-rw-r--	Azureus...
Microsoft.Extensions.D...	63 KB	18/06/2026 9:14:47 AM	rw-rw-r--	Azureus...
Microsoft.Extensions.D...	91 KB	18/06/2026 9:14:48 AM	rw-rw-r--	Azureus...
Microsoft.Extensions.D...	78 KB	17/06/2026 5:18:47 PM	rw-rw-r--	Azureus...
Microsoft.Extensions.L...	64 KB	18/06/2026 9:14:49 AM	rw-rw-r--	Azureus...

0 B of 7.00 MB in 0 of 34

SFTP-3 1:49:26

Outcome



STEP 9: CREATE SYSTEMD SERVICE

Aim

To run the ASP.NET Core application as a background service.

Katta siva shankar

Sivashankar9929@gmail.com

Procedure

Create Service File:

- `sudo nano /etc/systemd/system/electrogo.service`

Configure service.

Reload daemon:

- `sudo systemctl daemon-reload`

Enable Service:

- `sudo systemctl enable electrogo`

Start Service:

- `sudo systemctl start electrogo`

Check Status:

- `sudo systemctl status electrogo`

Screenshot

Insert Screenshot: Service File

Insert Screenshot: Service Running

Outcome

Application configured as Linux service.

STEP 10: CREATE MYSQL FLEXIBLE SERVER

Aim

To deploy managed MySQL database.

Procedure

1. Navigate to Azure Database for MySQL Flexible Server.
2. Click Create.
3. Configure:
 - a. Server Name
 - b. Admin Username

- c. Password
 - d. MySQL Version 8.0
- 4. Select Private Access.
- 5. Associate with VNet.
- 6. Create Server.

Screenshot

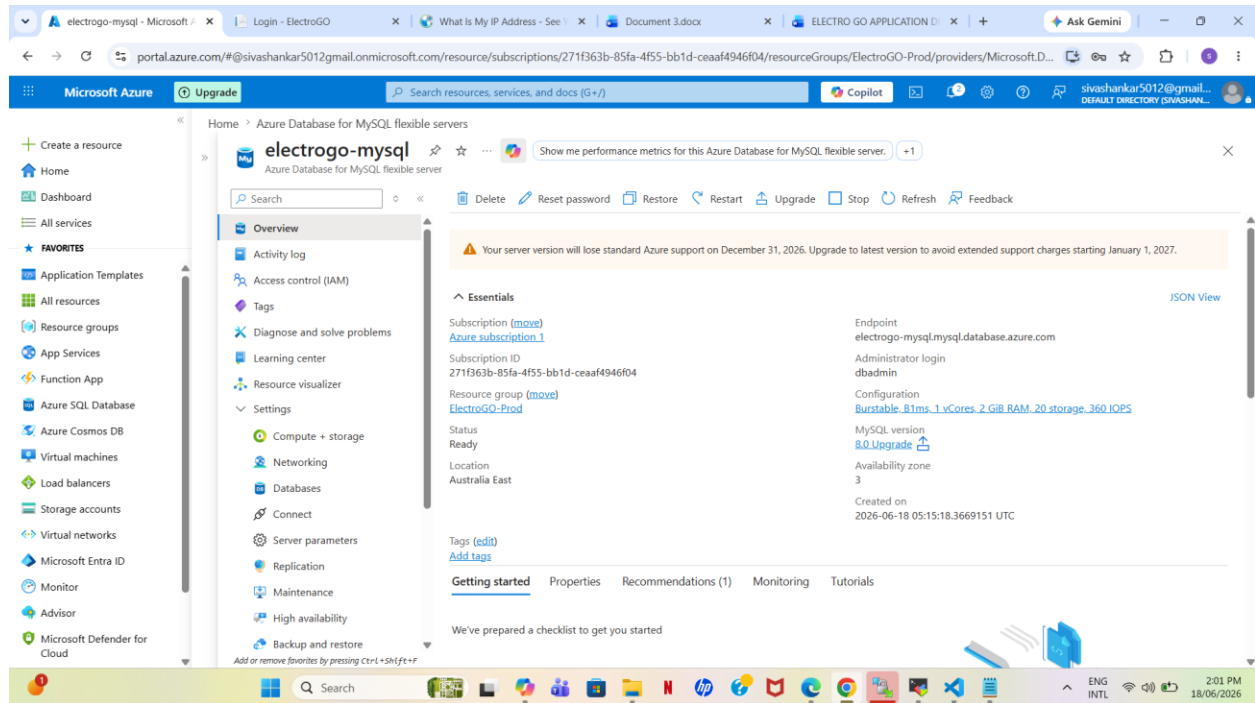
Microsoft Azure portal showing the configuration of an Azure Database for MySQL flexible server named 'electrogo-mysql'.

The top screenshot displays the 'Azure Database for MySQL flexible servers' overview page. The table below lists the server details:

Name	Type	Status	High availability	Location	Resource Group	Subscription
electrogo-mysql	Azure Database for ...	Ready	Disabled	Australia East	ElectroGO-Prod	Azure subscription 1

The bottom screenshot shows the 'Networking' configuration page for the 'electrogo-mysql' server. The configuration includes:

- Virtual network:** Subscription: Azure subscription 1, Virtual network: vnet-webapp, Subnet: subnet-private. A note states: "This subnet is delegated for use only with MySQL Flexible Server (Microsoft.DBforMySQL/flexibleServers)."
- Private DNS integration:** Subscription: Azure subscription 1, Private DNS zone: databaseservermysql-prod.private.mysql.database.azure.com. A note states: "Private DNS zone integration is required to connect to your Flexible Server in virtual network using server name (fully qualified domain name). A new private DNS zone will be created or you can optionally choose an existing one linked to the selected virtual network. With private DNS zone integration, the DNS records for the server name will be updated automatically in case the IP address of your Flexible Server changes."



Outcome

MySQL Flexible Server created successfully.

STEP 11: CONNECT APPLICATION TO MYSQL

Aim

To establish database connectivity.

Procedure

Install MySQL Client:

- `sudo apt install mysql-client -y`

Connect:

- `mysql -h .mysql.database.azure.com -u adminuser -p`

Create Database:

`CREATE DATABASE ElectroGoDB;`

Configure appsettings.json Connection String.

Run Migration:

- dotnet ef database update

Screenshot

```
root@WebApp-Server:/home/Azureuser# sudo apt install mysql-client -y
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
The following additional packages will be installed:
  mysql-client-8.0 mysql-client-core-8.0 mysql-common
The following NEW packages will be installed:
  mysql-client mysql-client-8.0 mysql-client-core-8.0 mysql-common
0 upgraded, 4 newly installed, 0 to remove and 26 not upgraded.
Need to get 2778 kB of archives.
After this operation, 61.8 MB of additional disk space will be used.
Get:1 http://azure.archive.ubuntu.com/ubuntu noble-updates/main amd64 mysql-client-core-8.0 amd64 8.0.46-0ubuntu0.24.04.2 [2740 kB]
Get:2 http://azure.archive.ubuntu.com/ubuntu noble/main amd64 mysql-common all 5.8+1.1.0build1 [6746 B]
Get:3 http://azure.archive.ubuntu.com/ubuntu noble-updates/main amd64 mysql-client-8.0 amd64 8.0.46-0ubuntu0.24.04.2 [22.5 kB]
Get:4 http://azure.archive.ubuntu.com/ubuntu noble-updates/main amd64 mysql-client all 8.0.46-0ubuntu0.24.04.2 [9414 B]
Fetched 2778 kB in 0s (14.9 MB/s)
Selecting previously unselected package mysql-client-core-8.0.
(Reading database ... 69353 files and directories currently installed.)
Preparing to unpack .../mysql-client-core-8.0_8.0.46-0ubuntu0.24.04.2_amd64.deb ...
Unpacking mysql-client-core-8.0 (8.0.46-0ubuntu0.24.04.2) ...
Selecting previously unselected package mysql-common.
Preparing to unpack .../mysql-common_5.8+1.1.0build1_all.deb ...
Unpacking mysql-common (5.8+1.1.0build1) ...
Selecting previously unselected package mysql-client-8.0.
Preparing to unpack .../mysql-client-8.0_8.0.46-0ubuntu0.24.04.2_amd64.deb ...
Unpacking mysql-client-8.0 (8.0.46-0ubuntu0.24.04.2) ...
Selecting previously unselected package mysql-client.
Preparing to unpack .../mysql-client_8.0.46-0ubuntu0.24.04.2_all.deb ...
Unpacking mysql-client (8.0.46-0ubuntu0.24.04.2) ...
Setting up mysql-common (5.8+1.1.0build1) ...
update-alternatives: using /etc/mysql/my.cnf.fallback to provide /etc/mysql/my.cnf (my.cnf) in auto mode
Setting up mysql-client-core-8.0 (8.0.46-0ubuntu0.24.04.2) ...
Setting up mysql-client-8.0 (8.0.46-0ubuntu0.24.04.2) ...
Setting up mysql-client (8.0.46-0ubuntu0.24.04.2) ...
Processing triggers for man-db (2.12.0-4build2) ...

Azureuser@WebApp-Server:~$ mysql -h electrogo-mysql.mysql.database.azure.com -u dbadmin -p
Enter password:
Welcome to the MySQL monitor.  Commands end with ; or \g.
Your MySQL connection id is 14
Server version: 8.0.44-azure Source distribution

Copyright (c) 2000, 2026, Oracle and/or its affiliates.

Oracle is a registered trademark of Oracle Corporation and/or its
affiliates. Other names may be trademarks of their respective
owners.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

mysql> CREATE DATABASE electrogo;
Query OK, 1 row affected (0.03 sec)

mysql> CREATE USER 'electroapp'@'%' IDENTIFIED BY 'Sky9Wave2026Lock';
Query OK, 0 rows affected (0.01 sec)

mysql> GRANT ALL PRIVILEGES ON electrogo.* TO 'electroapp'@'%';
Query OK, 0 rows affected (0.01 sec)

mysql> FLUSH PRIVILEGES;
Query OK, 0 rows affected (0.01 sec)

mysql> EXIT;
Bye
Azureuser@WebApp-Server:~$ mysql -h electrogo-mysql.mysql.database.azure.com -u electroapp -p electrogo -e "SELECT 1;"
Enter password:
+----+
| 1 |
+----+
| 1 |
+----+
```

```

azureuser@WebApp-Server:~$ pkill -9 -f ElectroGo.dll
cd /home/Azureuser/ElectroGo
nohup dotnet ElectroGo.dll --urls http://0.0.0.0:5000 > app.log 2>&1 &
sleep 3
cat app.log
[1] 248515
nohup: ignoring input
Hosting environment: Production
info: Microsoft.EntityFrameworkCore.Database.Command[20101]
      Executed DbCommand (12ms) [Parameters=[], CommandType='Text', CommandTimeout='30']
      SELECT CASE WHEN COUNT(*) = 0 THEN FALSE ELSE TRUE END
      FROM information_schema.tables
      WHERE table_type = 'BASE TABLE' AND table_schema = 'electrogo'
info: Microsoft.EntityFrameworkCore.Database.Command[20101]
      Executed DbCommand (12ms) [Parameters=[], CommandType='Text', CommandTimeout='30']
      ALTER DATABASE CHARACTER SET utf8mb4;
info: Microsoft.EntityFrameworkCore.Database.Command[20101]
      Executed DbCommand (72ms) [Parameters=[], CommandType='Text', CommandTimeout='30']
      CREATE TABLE `Users` (
        `Id` int NOT NULL AUTO_INCREMENT,
        `Username` longtext CHARACTER SET utf8mb4 NOT NULL,
        `Email` longtext CHARACTER SET utf8mb4 NOT NULL,
        `PasswordHash` longtext CHARACTER SET utf8mb4 NOT NULL,
        `FullName` longtext CHARACTER SET utf8mb4 NOT NULL,
        `Phone` longtext CHARACTER SET utf8mb4 NOT NULL,
        CONSTRAINT `PK_Users` PRIMARY KEY (`Id`)
      ) CHARACTER SET=utf8mb4;
info: Microsoft.Hosting.Lifetime[14]
      Now listening on: http://0.0.0.0:5000
info: Microsoft.Hosting.Lifetime[0]
      Application started. Press Ctrl+C to shut down.
info: Microsoft.Hosting.Lifetime[0]
      Hosting environment: Production
info: Microsoft.Hosting.Lifetime[0]
      Content root path: /home/Azureuser/ElectroGo

```

```

Azureuser@WebApp-Server:~/ElectroGo$ mysql -h electrogo-mysql.mysql.database.azure.com -u electroapp -p electrogo -e "SELECT Id, Username, Email, FullName FROM Users;"
Enter password:
Azureuser@WebApp-Server:~/ElectroGo$ mysql -h electrogo-mysql.mysql.database.azure.com -u electroapp -p electrogo -e "SELECT Id, Username, Email, FullName FROM Users;"
Enter password:
+----+-----+-----+-----+
| Id | Username | Email | FullName |
+----+-----+-----+-----+
| 1 | sivashankar2926@gmail.com | sivashankar2926@gmail.com | sivashankar |
+----+-----+-----+-----+
Azureuser@WebApp-Server:~/ElectroGo$ mysql -h electrogo-mysql.mysql.database.azure.com -u electroapp -p electrogo -e "SELECT Id, Username, Email, FullName FROM Users;"
Enter password:
+----+-----+-----+-----+
| Id | Username | Email | FullName |
+----+-----+-----+-----+
| 1 | sivashankar2926@gmail.com | sivashankar2926@gmail.com | sivashankar |
| 2 | goutham | goutham@gmail.com | goutham |
+----+-----+-----+-----+
Azureuser@WebApp-Server:~/ElectroGo$ mysql -h electrogo-mysql.mysql.database.azure.com -u electroapp -p electrogo -e "SELECT Id, Username, Email, FullName FROM Users;"
Enter password:
+----+-----+-----+-----+
| Id | Username | Email | FullName |
+----+-----+-----+-----+
| 1 | sivashankar2926@gmail.com | sivashankar2926@gmail.com | sivashankar |
| 2 | goutham | goutham@gmail.com | goutham |
| 3 | maheshbabu | maheshbabu@gmail.com | mahesh babu |
| 4 | vineeth | vineeth@gmail.com | vineeth |
+----+-----+-----+-----+

```

Outcome

Application successfully connected to MySQL database.

STEP 12: CREATE AZURE LOAD BALANCER

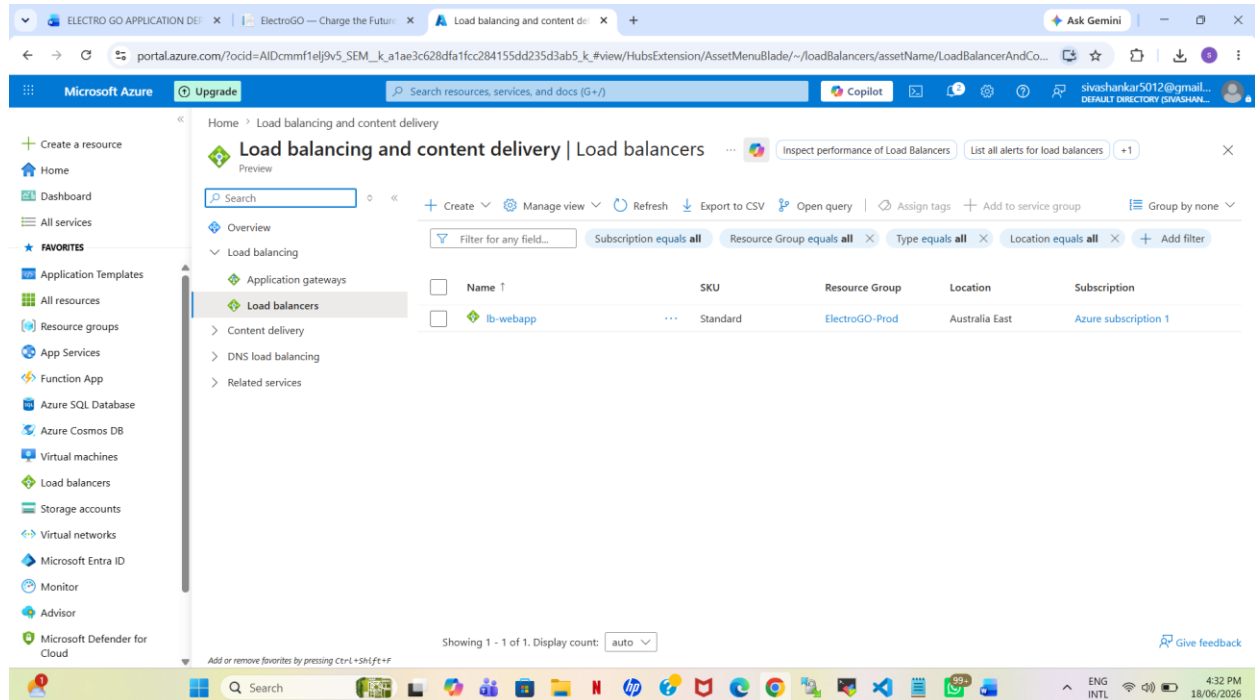
Aim

To distribute incoming traffic to the application.

Procedure

1. Create Public Load Balancer.
2. Create Public IP.
3. Create Backend Pool.
4. Add VM.
5. Configure Health Probe.
6. Configure Load Balancing Rule.

Screenshot



Outcome

Load Balancer configured successfully.

FINAL VALIDATION

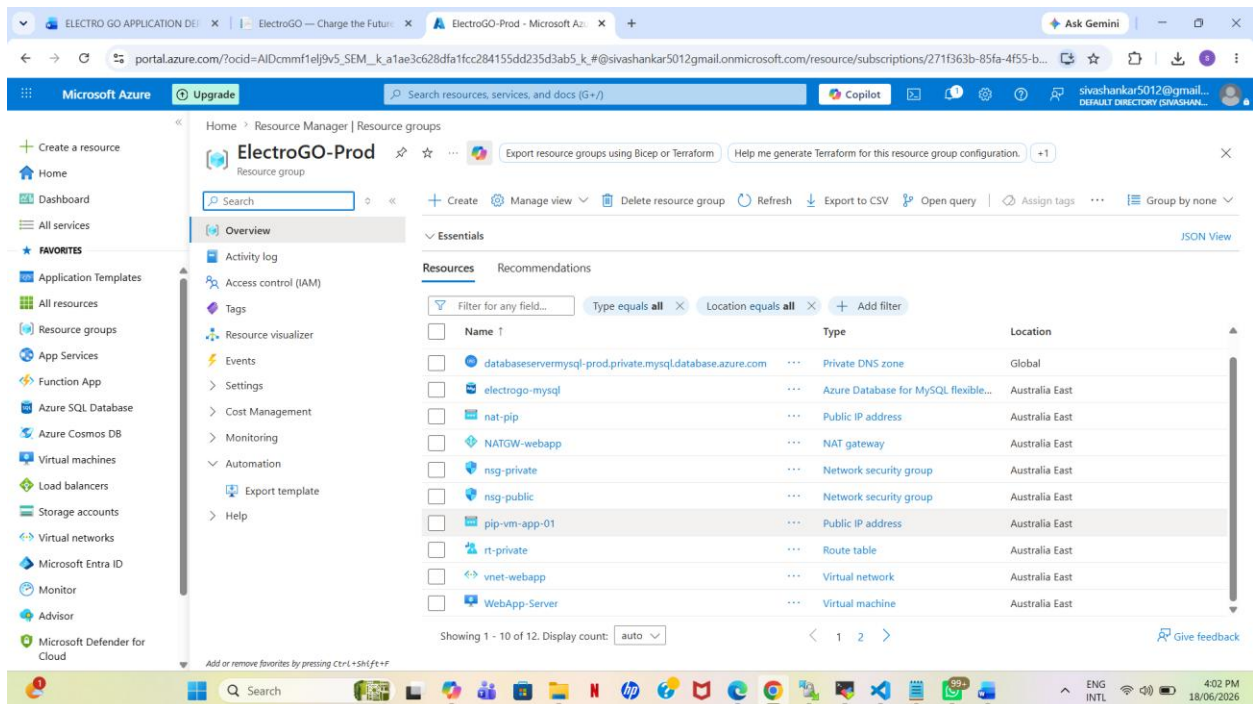
Aim

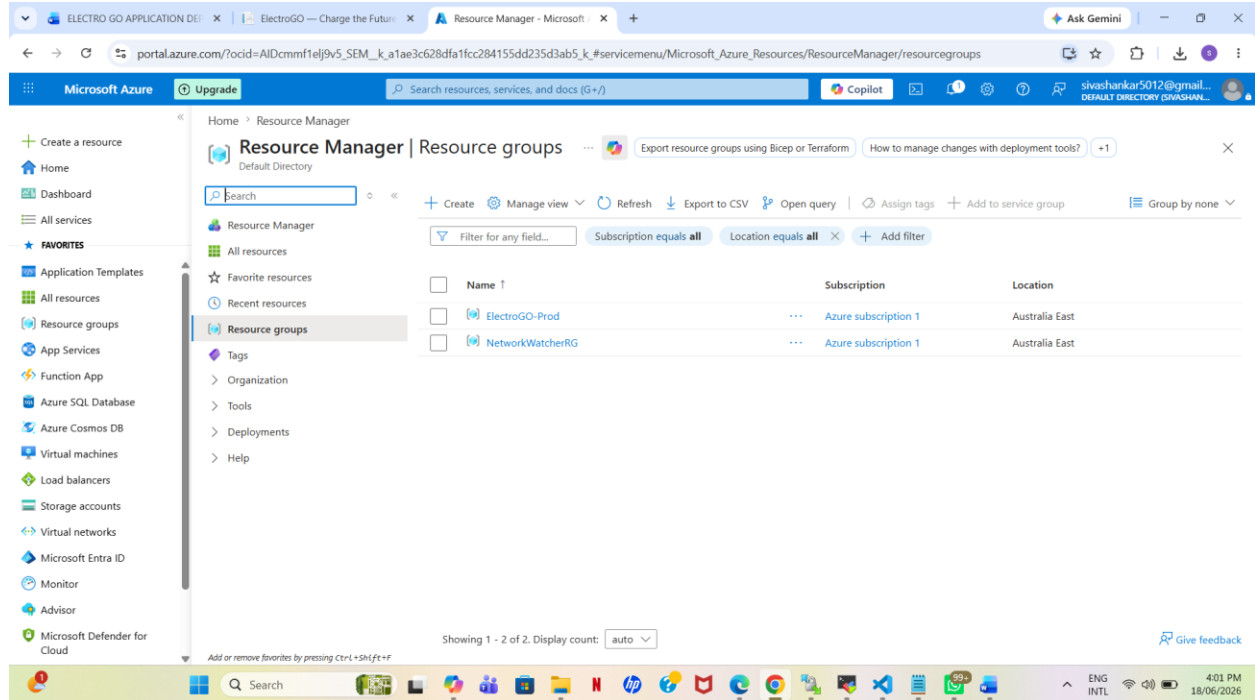
To verify successful deployment.

Procedure

1. Open Resource Group.
2. Verify VNet.
3. Verify VM.
4. Verify MySQL Server.
5. Verify NAT Gateway.
6. Verify NSGs.
7. Verify Load Balancer.
8. Access Application using:

Screenshot





Outcome

Electro GO Application was successfully deployed on Microsoft Azure using a secure 2-Tier Architecture consisting of Ubuntu VM (Application Tier) and Azure Database for MySQL Flexible Server (Database Tier).